Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

HWSW-Systeme

Übung 13: FPGA Elemente: BRAM

M.Sc. Eike Trumann, M.Sc. Jasper Homann,

Prof. Dr.-Ing. Guillermo Payá Vayá

30. Januar 2026

1 Block Ram (BRAM)

Es soll eine Dual-Port Registerbank mit 32-bit breiten Daten realisiert werden. Die benötigte Speichertiefe ist 1024 Einträge. Dabei soll ein Port des Speichers für den Lesezugriff, der andere für den Schreibzugriff genutzt werden.

a) Beschreiben Sie den Speicher in VHDL

```
library IEEE;
use IEEE.std_logic_1164.all;
use ieee.numeric_std.all;


entity registerbank is
    generic (
        DATAW : natural :=       ;
        ADDRW : natural :=
    );
    port (





    );
end entity;
```

```
architecture rtl of registerbank is

end architecture;
```

b) Wie viele Block-RAM Primitives werden bei einer FPGA-Synthese für einen AMD/Xilinx Zynq-7000 AP SoC XC7Z020 benötigt?

c) Wie viele BRAM Primitives werden benötigt, wenn eine Datenbreite von 16-bit ausreicht?

d) Für eine Prozessorarchitektur wird eine Registerbank mit zwei Lese- und einem Schreibport benötigt. Skizzieren Sie ein Blockdiagramm, um diese Registerbank zu implementieren, das den in (a) beschriebenen Speicherblock instanziiert.

2 First-In-First-Out (FIFO)

2.1 BRAM-FIFO Instanzierung

Auf AMD/Xilinx FPGAs können FIFOs verwendet werden, die aus BRAM mit einer direkt daran angegliederten Hardwarelogik für die FIFO-Funktionalität bestehen. Diese ist vom Hersteller vorgegeben und unter dem Namen FIFO36E1 verfügbar.

Füllen Sie das Template, um ein synchrones FIFO mit einer Wortbreite von 32 Bit und einer maximale Tiefe von 1024 zu realisieren. Verwenden Sie keinen ECC und fügen sie ggf. weitere benötigte Signale und Logik hinzu.

```
library IEEE;
use IEEE.std_logic_1164.all;
use ieee.numeric_std.all;

Library UNISIM;
use UNISIM.vcomponents.all;

entity fifo_bram is
port (
    clk, rst_n          : in  std_ulogic;          -- clock and reset
    wr_i, rd_i          : in  std_ulogic;          -- write- and read enable
    data_i              : in  std_logic_vector(31 downto 0);
    full_o, empty_o     : out std_ulogic;          -- status flags of the FIFO
    data_o              : out std_logic_vector(31 downto 0)
);
end entity;

architecture structural of fifo_bram is




begin




-- FIFO36E1: 36Kb FIFO (First-In-First-Out) Block RAM Memory
--           7 Series
-- Xilinx HDL Language Template, version 2024.2
FIFO36E1_inst : FIFO36E1
generic map (
   ALMOST_EMPTY_OFFSET =>              , -- Sets the almost empty threshold
   ALMOST_FULL_OFFSET =>               , -- Sets almost full threshold
   DATA_WIDTH =>                       , -- Sets data width to 4-72
   DO_REG =>                           , -- Enable output register (1-0) Must be 1 if EN_SYN = FALSE
   EN_ECC_READ =>                      , -- Enable ECC decoder, FALSE, TRUE
   EN_ECC_WRITE =>                     , -- Enable ECC encoder, FALSE, TRUE
   EN_SYN =>                           , -- Specifies FIFO as Asynchronous (FALSE) or Synchronous (TRUE)
   FIFO_MODE =>                        , -- Sets mode to "FIFO36" or "FIFO36_72"
   FIRST_WORD_FALL_THROUGH =>          , -- Sets the FIFO FWFT to FALSE, TRUE
   INIT =>                             , -- Initial values on output port
   SIM_DEVICE =>                       , -- Must be set to "7SERIES" for simulation behavior
   SRVAL =>                           -- Set/Reset value for output port
)
port map (
   -- ECC Signals: 1-bit (each) output: Error Correction Circuitry ports
   DBITERR =>                          , -- 1-bit output: Double bit error status
   ECCPARITY =>                        , -- 8-bit output: Generated error correction parity
   SBITERR =>                          , -- 1-bit output: Single bit error status
   -- Read Data: 64-bit (each) output: Read output data
   DO =>                               , -- 64-bit output: Data output
   DOP =>                              , -- 8-bit output: Parity data output
```

```
   -- Status: 1-bit (each) output: Flags and other FIFO status outputs
   ALMOSTEMPTY =>                    , -- 1-bit output: Almost empty flag
   ALMOSTFULL =>                     , -- 1-bit output: Almost full flag
   EMPTY =>                          , -- 1-bit output: Empty flag
   FULL =>                           , -- 1-bit output: Full flag
   RDCOUNT =>                        , -- 13-bit output: Read count
   RDERR =>                          , -- 1-bit output: Read error
   WRCOUNT =>                        , -- 13-bit output: Write count
   WRERR =>                          , -- 1-bit output: Write error
   -- ECC Signals: 1-bit (each) input: Error Correction Circuitry ports
   INJECTDBITERR =>                  , -- 1-bit input: Inject a double bit error input
   INJECTSBITERR =>                  ,
   -- Read Control Signals: 1-bit (each) input: Read clock, enable and reset input signals
   RDCLK =>                          , -- 1-bit input: Read clock
   RDEN =>                           , -- 1-bit input: Read enable
   REGCE =>                          , -- 1-bit input: Clock enable
   RST =>                            , -- 1-bit input: Reset
   RSTREG =>                         , -- 1-bit input: Output register set/reset
   -- Write Control Signals: 1-bit (each) input: Write clock and enable input signals
   WRCLK =>                          , -- 1-bit input: Rising edge write clock.
   WREN =>                           , -- 1-bit input: Write enable
   -- Write Data: 64-bit (each) input: Write input data
   DI =>                             , -- 64-bit input: Data input
   DIP =>                            , -- 8-bit input: Parity input
);
-- End of FIFO36E1_inst instantiation

end architecture;
```

3 Stack (LIFO)

Implementieren Sie einen synchronen Stack (Last-In-First-Out (LIFO)) Speicher mit einer Tiefe von 1024 Einträgen mit einer Breite von jeweils 32-bit. Der Speicher soll je einen dedizierten Read- und Write-Port haben. Bei gleichzeitigem Zugriff auf beide Ports soll zuerst der oberste Wert gelesen und dann überschrieben werden (Read-First).

Im vollen bzw. leeren Zustand soll kein Schreib bzw. Lesevorgang erfolgen und ein `full_o` bzw. `empty_o` Signal ausgegeben werden. Die Unterscheidung des vollen und leeren Zustands soll über die ein zusätzliches Bit in der Addressierungslogik erfolgen.

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity LIFO is
    generic (
        DATA_WIDTH : natural := 32;
        ADDR_WIDTH : natural := 10
    );
    port (

    );
end entity;
```

```
architecture rtl of LIFO is









begin

    ram_proc: process(clk_i) is
    begin
```

```
    end process;

    seq: process(clk_i) is
    begin
```

```
    end process;

    empty <=

    full <=

    addr_logic: process (all) is
    begin
```

```
    end process;

end architecture;
```